Quattro LED sulla breadboard: la luce che scorre

Lezione 9 — 1MAK, SST-INF-01c — Circuito ordinato su breadboard, colori dei fili, setup e loop scritti da zero, sequenza di LED, leggere gli errori

03/11/2026

1 Da dove ripartiamo

L'altra volta abbiamo collegato ad Arduino **un LED** e poi **quattro LED** "in aria", e abbiamo conosciuto la **breadboard**. Oggi rifacciamo il circuito con i quattro LED **sulla breadboard** e scriviamo il programma **da zero**, per far scorrere la luce da un LED all'altro.

Nota

Per entrare in Tinkercad, il browser dev'essere collegato al vostro account della scuola. In alto a destra, **vicino alla X di chiusura**, può comparire il pulsante **Verifica la tua identità**: va cliccato per fare il login. Non basta entrare dall'icona dentro il sito. Tinkercad si scrive **con la K**.

2 Il circuito

Si parte da un nuovo circuito con un **Arduino Uno** e la **breadboard piccola**. I collegamenti vanno fatti con criterio: in elettronica non esiste **"a caso"**. Copiate esattamente il disegno.

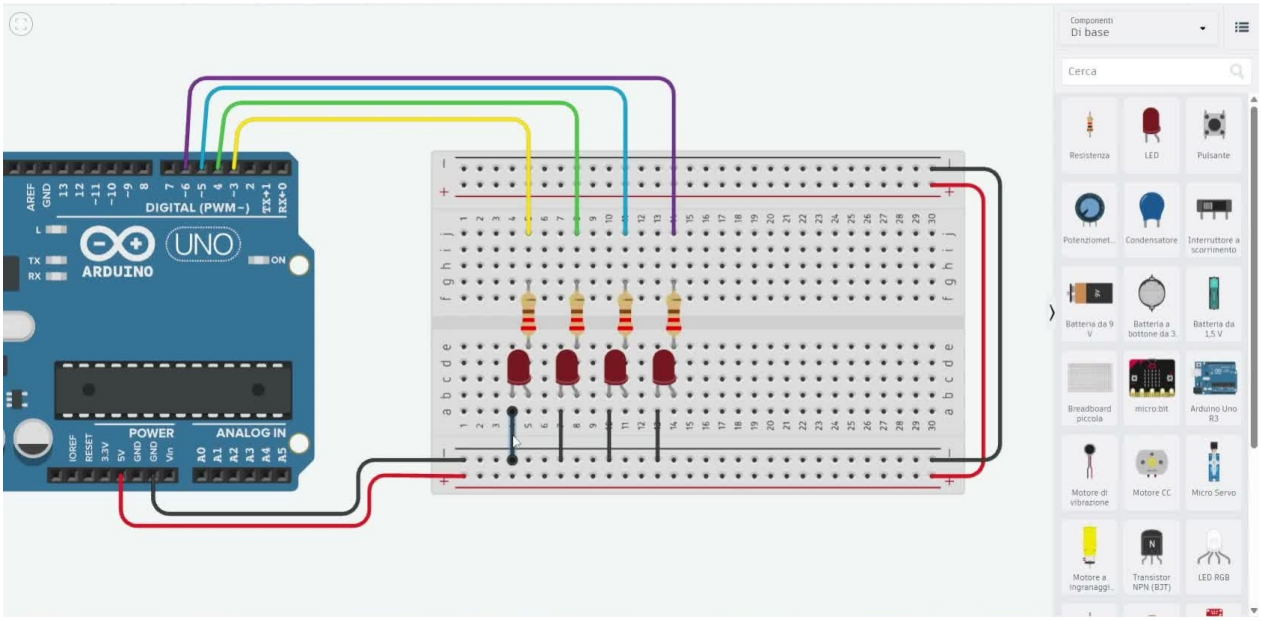


Figura 1: Il circuito da realizzare: quattro fili colorati dai pin 3, 4, 5 e 6, quattro resistori a cavallo del solco centrale, quattro LED, i fili neri verso il rail del meno.

1. Filo **nero** da **GND** di Arduino al rail $-$ in basso; filo **rosso** da **5V** al rail $+$ in basso.
2. Sulla destra della breadboard, un filo rosso e un filo nero collegano i rail in basso a quelli **in alto**. Non dimenticateli.
3. Fili dai **pin 3, 4, 5, 6** (giallo, verde, azzurro, viola) alle **colonne 5, 8, 11, 14**, nella riga j della parte alta: non sul rail rosso, ma sulle colonne.
4. Nelle stesse colonne, quattro **resistori** da **220 Ω** a cavallo del solco centrale, dalla riga g alla riga e.
5. Sotto, quattro **LED** nella riga b, con l'**anodo** nella colonna del resistore (5, 8, 11, 14) e il **catodo** nella colonna a sinistra (4, 7, 10, 13). Portando il mouse su un piedino del LED, Tinkercad mostra la scritta **Anodo** o **Catodo**.
6. Dal **catodo** di ogni LED, un filo **nero** dalla riga a fino al rail $-$.

Attenzione

Attenzione ai **dettagli**: i resistori sono da **220 Ω**, non da 1 kΩ (e non 1220). Controllate i colori delle fasce: **rosso, rosso, marrone**. Scrivete il valore e scegliete l'unità Ω nel menu.

Nota

Il colore dei fili è uno standard. I fili di **GND**, cioè il meno, vanno sempre **neri**. Il filo del **5V** va sempre **rosso**. Non è che se usate un altro colore scoppia qualcosa: è la **convenzione**. Quando aprite un circuito elettronico e vedete un filo nero e uno rosso, sapete che quelle sono le **alimentazioni**, cioè le tensioni stabili. Per gli altri fili scegliete voi, **evitando nero e rosso**.

Nota

Fili storti. Se un filo ha dei punti in mezzo, come una spezzata, selezionate il singolo punto e premete **Canc**: il filo torna dritto. In Tinkercad si seleziona e si cancella anche un filo intero messo per sbaglio.

3 Il codice, scritto da capo

Si preme **Codice** e dai **blocchi** si passa a **Testo**. Tinkercad propone un programma già pronto (`C++ code`, con `LED_BUILTIN` e `delay(1000)`): cancelliamo tutto e riscriviamo quello che ci serve.

3.1 I due blocchi fondamentali

Per funzionare, Arduino ha bisogno di **due blocchi di codice**. Un **blocco** è qualcosa che inizia con la parola `void`, ha un nome (`setup` oppure `loop`), due parentesi tonde e le **graffe**. Tutto quello che sta dentro le graffe fa parte di quel blocco.

Definizione

`void setup()` viene eseguito **una sola volta**, all'avvio della macchina. Di solito contiene le **impostazioni**.
`void loop()` viene **ripetuto infinite volte**, come dice il nome. Contiene le **operazioni** da ripetere.

Nota

Come si scrivono le graffe su una tastiera italiana: tasto **AltGr** + **Maiuscolo** + **è** per la graffa aperta `{`, tasto **AltGr** + **Maiuscolo** + **+** per la graffa chiusa `}`. Sono gli stessi tasti delle parentesi quadre, che si fanno senza Maiuscolo. La `S` di `setup` è minuscola.

3.2 Dentro setup: pinMode

Tra le impostazioni c'è una cosa che si fa sempre: dire se un pin (o terminale, o piedino: è la stessa cosa) è un **ingresso** o un'**uscita**. È la macchina che dice ai LED se accendersi o spegnersi, quindi la direzione va **dalla macchina verso l'esterno**: sono tutte **uscite**. Per ciascun pin si usa `pinMode`, con il numero del pin e la parola `OUTPUT`:

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
}
```

La corrente **uscirà** dal pin. Se avessimo collegato un **pulsante**, per acquisire l'informazione che qualcuno lo preme, avremmo scritto `INPUT`. Per ora sono tutti `OUTPUT`.

Attenzione

Le **maiuscole** contano: la **M** di `pinMode` e tutto `OUTPUT`. Scritti in minuscolo, la macchina non li riconosce.

3.3 Dentro loop: accendere un LED alla volta

Ora vogliamo accendere **un LED alla volta**: il 3, poi il 4, il 5, il 6, con un'attesa di **200 millisecondi** per ogni passo. Servono due comandi:

- `digitalWrite(pin, valore)`: `HIGH` accende, `LOW` spegne, **tutto maiuscolo**;
- `delay(200)`: dice alla macchina di fermarsi per 200 millesimi di secondo.

Il ragionamento: accendo il 3 e aspetto. Poi spengo il 3 e accendo il 4, e aspetto. Poi spengo il 4 e accendo il 5, e aspetto. Poi spengo il 5 e accendo il 6, e aspetto. Infine spengo il 6: il `loop` ricomincia da capo e riaccende il 3.

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
}

void loop()
{
  // LOW spegne
  digitalWrite(3, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(4, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(5, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(6, LOW);
}
```

Scritto il codice, si preme **Avvia simulazione**. Per modificarlo bisogna prima premere **Interrompi simulazione**.

Nota

Niente attesa tra lo spegnimento e l'accensione successiva. Se mettete un `delay` anche dopo ogni `LOW`, il programma funziona, ma la luce si spegne, passa un momento di buio e solo dopo si accende il LED seguente. Come è scritto qui, tra lo spegnimento del 3 e l'accensione del 4 non c'è attesa, e il movimento è più **fluido**.

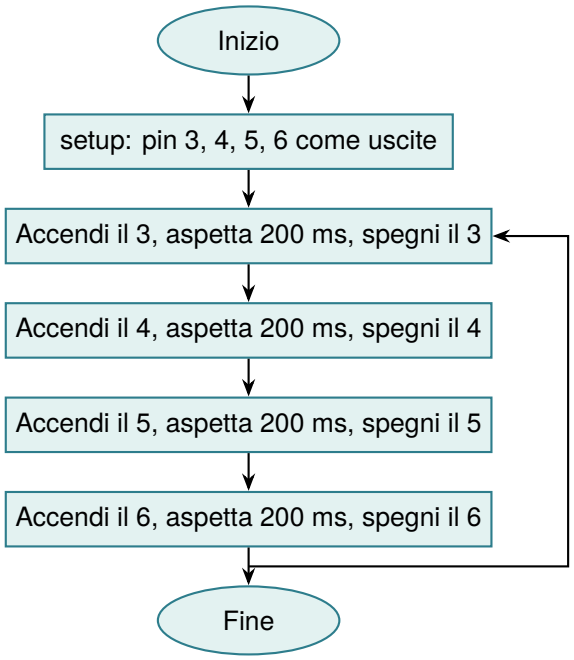


Figura 2: La sequenza: `loop` riparte sempre dal LED 3. La scheda non arriva mai davvero alla Fine.

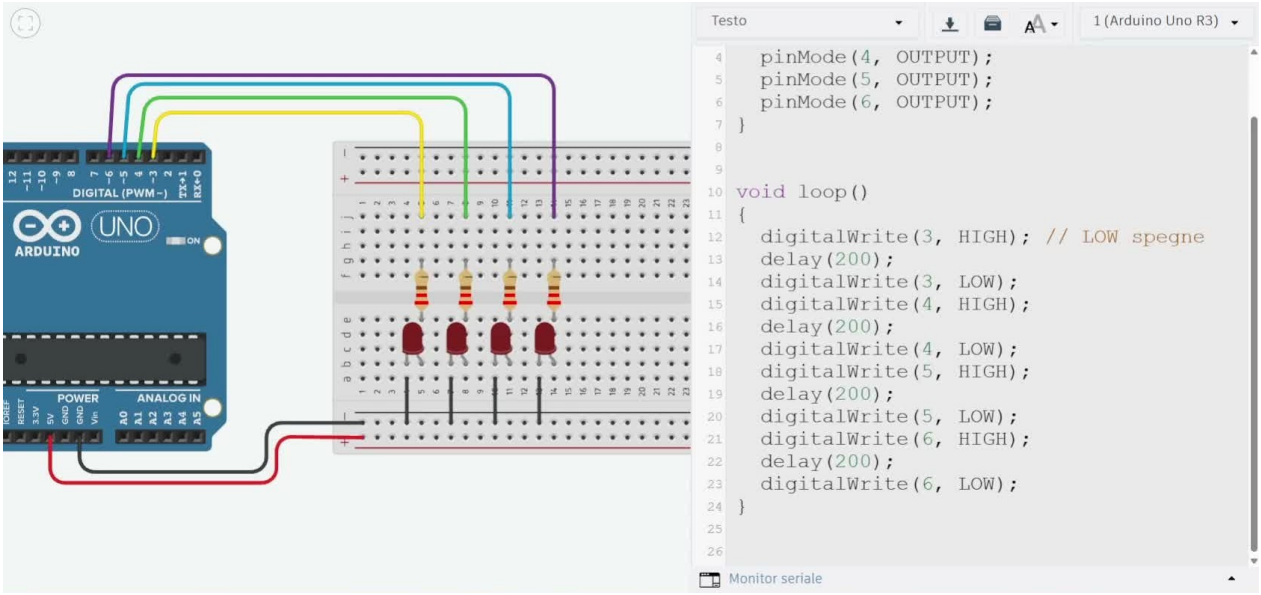


Figura 3: Il risultato in Tinkercad, con il codice accanto al circuito.

4 Il passo successivo: otto LED, avanti e indietro

Chi ha finito passa a **8 LED**: se ne aggiungono altri quattro **in fianco**, sui **pin 7, 8, 9, 10**, ciascuno con il suo resistore. La luce deve scorrere da sinistra verso destra e poi **tornare indietro**. Il setup deve dire che anche i pin 7, 8, 9 e 10 sono uscite.

Attenzione

I **fili rossi** sono riservati alle alimentazioni: per i nuovi fili di segnale usate altri colori, come il rosa, ed evitate il nero e il rosso.

Per tornare indietro non serve niente di nuovo: si ripetono le stesse operazioni **al contrario**. Dopo il pin 10 si accende il 9, poi l'8, fino al 4. Non serve riaccendere il 3 alla fine, perché lo fa il `loop` ripartendo da capo.

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(6, OUTPUT);
  pinMode(7, OUTPUT);
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
}

void loop()
{
  // la luce va avanti, dal 3 al 10
  digitalWrite(3, HIGH);
  delay(200);
  digitalWrite(3, LOW);
  digitalWrite(4, HIGH);
  delay(200);
  digitalWrite(4, LOW);
  digitalWrite(5, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(6, LOW);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(7, LOW);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);
  delay(200);
  digitalWrite(9, LOW);
  digitalWrite(10, HIGH);
  delay(200);
  digitalWrite(10, LOW);
  // la luce torna indietro, dal 9 al 4
  digitalWrite(9, HIGH);
  delay(200);
  digitalWrite(9, LOW);
  digitalWrite(8, HIGH);
  delay(200);
  digitalWrite(8, LOW);
  digitalWrite(7, HIGH);
  delay(200);
  digitalWrite(7, LOW);
  digitalWrite(6, HIGH);
  delay(200);
  digitalWrite(6, LOW);
  digitalWrite(5, HIGH);
  delay(200);
  digitalWrite(5, LOW);
  digitalWrite(4, HIGH);
  delay(200);
  digitalWrite(4, LOW);
}
```

5 Quando qualcosa non va

Il simulatore **dice cosa non va**: guardate **sotto il codice**, dove in rosso compare l'errore con il **numero di riga**. Leggetelo prima di chiedere.

- `expected ';' before ...`: **expected** vuol dire "mi sarei aspettato" e **before** vuol dire "prima di". Manca il **punto e virgola** alla fine della riga prima, spesso dopo un `delay`: nel programma

qui sopra, per esempio, alle righe dei `delay`. Ogni comando finisce con `;`.

- `digitalWrite` scritto male: "digital right" non esiste. La **W** è maiuscola, il resto minuscolo, e si scrive **Write**.
- `LOW` va scritto **tutto maiuscolo**.
- Tutto rosso dopo `loop`: mancano le **parentesi tonde** dopo il nome.
- Una **graffa** di troppo o fuori posto: controllate la riga indicata.
- Due `setup`: ce ne deve essere **uno solo**. Non si duplica: si aggiungono le righe in quello che c'è già. Lo stesso vale per `loop`.
- Il LED nuovo non si accende: manca il suo `pinMode` nel setup, oppure i **fili nero e rosso** dei rail sono collegati male. Confrontate il disegno pezzo per pezzo.
- La luce va al contrario: controllate quale pin avete collegato a quale colonna.
- La simulazione sembra ferma: guardate il **Tempo simulatore** in alto, se avanza sta funzionando. A volte serve qualche secondo. Se la pagina non si aggiorna, abbassate la finestra e riapritela.

Nota

Il progetto si **salva da solo**. In alto gli si può dare un **nome** a piacere; poi si può chiudere e ritrovarlo intatto.

6 Il compito per casa

Il compito è il **C02**, con il testo a parte: sul circuito da **8 LED**, due luci che partono dai lati, vanno verso il **centro**, "sbattono" e **tornano indietro**. La consegna è uno **screenshot** della pagina su Classroom, **senza link**: per valutare bastano il circuito e il codice.

7 Cosa ricordare di questa lezione

- Copiare il circuito con **criterio**: GND **nero**, 5V **rosso**, resistori da **220 Ω**.
- Un **blocco** è `void`, un nome, le parentesi tonde e le graffe.
- `setup` si esegue **una volta** e contiene le impostazioni; `loop` si **ripete** all'infinito.
- `pinMode(pin, OUTPUT)` per ogni pin usato; `digitalWrite(pin, HIGH)` accende, `LOW` spegne; `delay(ms)` aspetta.
- Per far scorrere la luce: spegni il LED corrente e accendi il successivo, **senza attesa** in mezzo, poi aspetta.
- Gli errori si leggono **sotto il codice**: riga e messaggio. **expected** vuol dire "mi aspettavo".